

What Inputs Drive Effective LLM-based Unit Test Generation?

Saarang Agarwal, *University of Waterloo, Waterloo, ON, N2L 3G1, Canada*

Pengyu Nie, *University of Waterloo, Waterloo, ON, N2L 3G1, Canada*

Meiyappan Nagappan, *University of Waterloo, Waterloo, ON, N2L 3G1, Canada*

Abstract—Large Language Models (LLMs) have revolutionized software engineering by automating critical tasks such as code and test generation. The effectiveness of LLM-generated tests, however, largely depends on the quality and completeness of the input context—particularly the availability of software requirements and the correctness of code under test. We study five state-of-the-art LLMs at varying scales, investigating their capabilities in generating unit test cases while focusing on how different inputs impact test correctness, bug detection capability, and code coverage. Our findings reveal that, among the LLMs studied, combining code with clear requirements produces the highest-quality test cases, whereas generating tests from incorrect code alone yields significantly poorer results.

Software testing consumes a staggering 50% of total development costs and over 40% of resources in the software development life-cycle.¹ Despite advances in test automation, testing still represents one of the most expensive aspects of software engineering, requiring substantial resources and effort to ensure comprehensive coverage and quality. Large Language Models (LLMs) have emerged as game-changers for test automation: trained on the corpus of software code and tests, LLMs can automatically generate high-quality executable test cases, dramatically enhancing developers' productivity in testing and finding bugs.² Yet a critical question remains unanswered: how does the context (code and natural language requirements) provided to LLMs impact the quality of the tests they generate?

Most research studies on LLM-based test generation (e.g., CodaMOSA³ and CoverUp⁴) optimize for code coverage. Code coverage, while a great proxy, may not directly correlate with bug detection capability.⁵ Additionally, these studies and commercial tools like GitHub Copilot⁶ and Qodo⁷ focus on generating tests directly from code, leaving a rich source of information—natural language requirements from task descriptions and documentation—underutilized. While

requirements could be used with these tools, we do not know if they will be useful. We address this gap with a comprehensive experiment that considers the correctness of code and usage of requirements.

We introduce a framework for evaluating the effectiveness of LLM-based test generation techniques with varying inputs, and apply it on two closed-source and three open-source LLMs using CodeChef dataset.⁸ The focus of our study is to systematically investigate how the inclusion of different inputs—requirements, correct code, incorrect code, and their combinations—affect the quality of generated test cases across three key metrics: accuracy, bug detection capability, and code coverage.

A key finding of our study is that when the correctness of code is unknown, including requirements helps stabilize LLMs' performance, i.e., generating tests of same or higher quality. If code is likely incorrect (e.g., when using LLMs to generate bug-revealing tests), relying solely on requirements is the most effective approach. We also observed smaller models struggle to infer specifications and input/output constraints from natural language requirements, leading to suboptimal results when generating test cases with requirements in the inputs.

A contemporary study on the influence of incorrect code on test generation⁹ (on arXiv at the time of this submission) suggests that using correct code and including requirements lead to higher quality tests. Our

work validates and extends the study to the most state-of-the-art LLMs and a challenging dataset with varying complexity levels and input/output constraints. Furthermore, our framework is the first to integrate test case reduction to standardize the test cases generated from LLMs with varying output verbosity. We also adopt human-written incorrect code submissions rather than LLM-synthesized ones to improve the validity of the bug detection capability measurement. We examine the impact of task difficulty on test case quality and use Mann-Whitney test with p-values and effect sizes as additional data-points to analyze results.

EXPERIMENTAL DESIGN

This section outlines the key components of our study: dataset, LLM input construction, test generation and reduction processes, and evaluation metrics. This forms a comprehensive evaluation framework for LLM-based test generation, as illustrated in Figure 1.

Dataset

CodeChef (<https://www.codechef.com/>) is a widely recognized competitive programming platform offering coding problems spanning a diverse range of difficulty levels. Each problem comes with natural language requirements including a task description, input/output formats and constraints, and sometimes sample input/output pairs. User submitted code is executed against a comprehensive test suite (private to CodeChef) to check for correctness. All user submissions, correct or incorrect, are publicly visible, making it ideal for our study.

We follow a similar procedure to a recent study on LLMs for test-driven development⁸ to process the CodeChef dataset, using the same 1,100 problems—100 randomly sampled from 11 difficulty levels (focusing on Python).

For each problem, we extract the following information:

- requirement;
- one correct code submission (verified against CodeChef's test suite);
- three distinct incorrect code submissions (that fail on some test cases in CodeChef's test suite);
- metadata including the problem ID, difficulty score, and difficulty level.

To challenge LLMs in generating bug-revealing tests, we focus on partially correct submissions (labeled “wrong solution” by CodeChef) that pass syntax checks but contain logical flaws. We extract three such

submissions per problem to ensure bug diversity. We found four problems with fewer than three incorrect-code submissions and replaced them with other problems of similar difficulty from CodeChef.

LLM Inputs Construction

To study how different inputs affect LLMs' test generation quality, we design nine research cases (RCs), each of which constructs the LLM prompt with certain (combination of) information extracted from the CodeChef dataset for generating unit test cases:

- R: requirements only;
- CC: correct code only;
- IC₁, IC₂, IC₃: incorrect code only, using the first/second/third extracted incorrect code submission respectively;
- R-CC: requirements + correct code;
- R-IC₁, R-IC₂, R-IC₃: requirements + incorrect code, using the first/second/third extracted incorrect code submission respectively.

Determining the correctness of a given code is challenging. Thus, our experimental setup generates a range of results to highlight areas for model improvement and optimal usage strategies. Details are provided in the Results section.

Following the best practices in prompting LLMs, we prepare a system prompt and a user prompt for each RC. The system prompt defines the model's behavior and guidelines, clarifies the task (generating unit tests), and standardizes the output format for automated evaluation.

The system prompt is shown below, where the variable $\${provided_info_title}$ specifies what information will be provided as inputs, e.g., “programming requirements and corresponding python code” for R-CC.

```
You are an expert Python Programmer and QA tester.
Your job is to look at the  $\${provided\_info\_title}$ 
provided by the user and output a comprehensive
list of test cases for it.
Do not attempt to explain your output; simply
provide correct test cases (including edge cases).
The test cases should be in the format of:
    assert main(input1) == "expected_output1"
    assert main(input2) == "expected_output2"
Note that the function name in the assert statement
will always be main and the input to main will be
a string that will be used as a standard input
stream. There should be NO comments.
```

The user prompt is used to provide information based on the specific RC, reflecting the user's request and context during inference. For example, the user prompt for R-CC simply contains the requirements and correct code, as shown below:

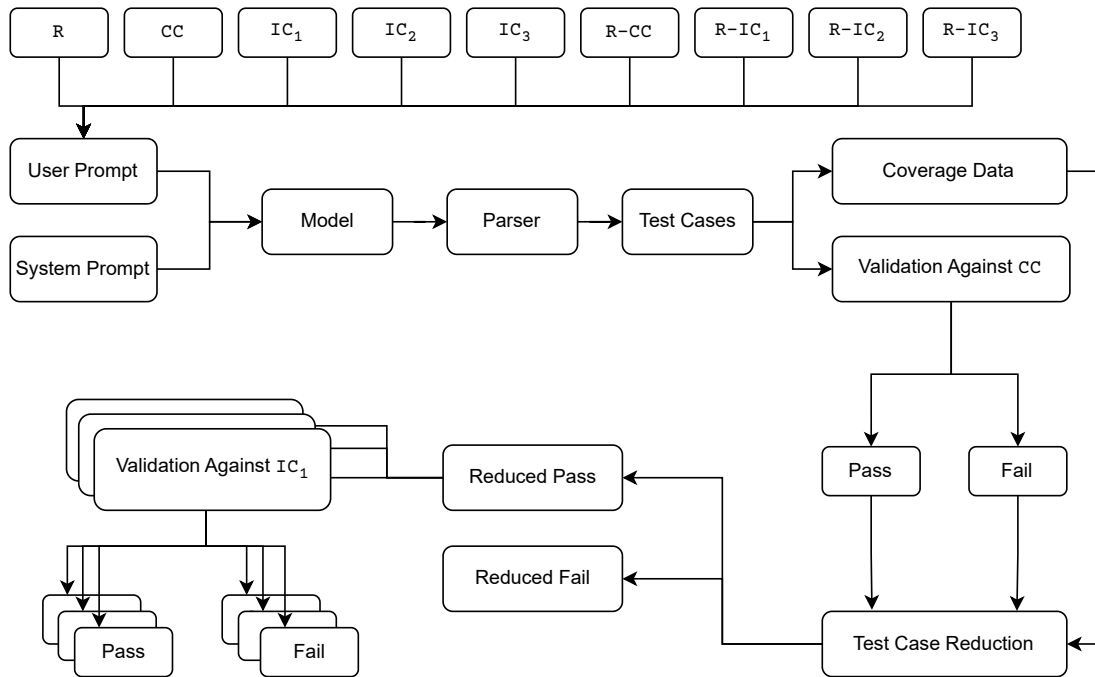


FIGURE 1: Workflow of our LLM-based test generation evaluation framework. Prompts containing different information form nine different research cases (requirements (R), correct code (CC), first incorrect code (IC₁), second incorrect code (IC₂), third incorrect code (IC₃), requirements and correct code (R-CC), requirements and first incorrect code (R-IC₁), requirements and second incorrect code (R-IC₂), and requirements and third incorrect code (R-IC₃)). These inputs are used to create the user prompt, which along with the system prompt is used by the LLM to generate test cases. These test cases are executed against the correct code to obtain pass/fail and code coverage data. This information is used for test case reduction and obtaining relevant evaluation metrics.

```

The requirements are:
${requirement}

and the corresponding Python code is:
${correct_code}

```

Test Generation and Reduction

Both prompts are passed to the LLM for inference using zero-shot prompting. The LLMs are hosted on Nvidia RTX 6000 Ada GPUs using Ollama (which adjusts the special tokens configurations specific to the LLM). We perform greedy decoding by setting the temperature to 0, which leads to high-quality outputs and also ensures deterministic and reproducible evaluation. The maximum token limit is set to 5,000 (~3,750 words), which is sufficient for the LLMs to generate comprehensive test cases.

Our prompts guide the LLM to generate test cases in a specific format. We create a parser that extracts these tests using pattern matching and auto-corrects minor formatting errors (e.g., removing comments, text, code block delimiters). If the model output deviates

significantly (e.g., natural language test plans), it's discarded as unusable.

After generating test cases, we validate them against the correct code with a pass/fail outcome and use Python's coverage.py module to measure code coverage.

In our preliminary experiments, we found that LLMs may generate similar or redundant tests that don't improve code coverage, potentially skewing results. To address this, we use a greedy algorithm¹⁰ for coverage-based test case reduction, removing tests that don't contribute to coverage increase. Note that we first group tests by outcome (pass/fail), and then perform reduction separately to each group (reduced pass/fail). This ensures a fair comparison of tests across LLMs at different verbosity levels. For example, if an LLM generates 10 tests, 9 passing but covering the same code-paths and 1 failing, without reduction, this would suggest 90% accuracy. After reduction, we keep just two tests, reflecting a more realistic 50% accuracy.

Evaluation Metrics

We evaluate quality of the generated test cases, after test case reduction, using three metrics.

Accuracy measures how well the LLM generates tests that pass against correct code, reflecting its ability to understand code and/or requirements and create syntactically and semantically valid tests. For a given RC, the accuracy (A_{RC}) is calculated as the average percentage of reduced correct test cases over the dataset:

$$A_{RC} = \frac{\sum_{i=1}^N \frac{c_{RC,i}}{c_{RC,i} + f_{RC,i}}}{N} * 100 \quad (1)$$

where $c_{RC,i}$, $f_{RC,i}$ is the number of reduced correct/incorrect test cases for problem i under the RC, and N is the dataset size (1,100).

For RCs involving multiple incorrect code versions (IC and R-IC), we average results across the three versions (i.e., $A_{IC} = (A_{IC_1} + A_{IC_2} + A_{IC_3})/3$ and $A_{R-IC} = (A_{R-IC_1} + A_{R-IC_2} + A_{R-IC_3})/3$) since there is no inherent ordering between them.

Bug Detection Capability evaluates how effectively the generated test cases can identify bugs in incorrect code. A high-quality test suite should pass on correct code while failing on buggy code. The bug detection capability for a given RC (B_{RC}) is calculated as:

$$B_{RC} = \frac{\sum_{i=1}^N \sum_{j=1}^3 \frac{\text{sgn}(g_{RC,i,IC_j})}{c_{RC,i}}}{3 * N} * 100 \quad (2)$$

where sgn is the sign function (returning 1 when input is positive, 0 otherwise), g_{RC,i,IC_j} represents the number of reduced correct test cases that fail when executed against incorrect implementation IC_j for problem i . The usage of sgn implies that to consider a bug being detected by a test suite, it is sufficient for at least one test case to fail on the incorrect code. Similar to accuracy, we average the results for RCs involving multiple incorrect code versions.

Code Coverage measures how thoroughly the generated tests exercise the implementation code. We measure line-level code coverage using Python's `coverage.py` module, which is the percentage of lines covered by the reduced correct test cases divided by the total number of lines in the correct code. The code coverage for a given RC (C_{RC}) is calculated as the average line coverage on each problem in the dataset, similar to how accuracy is calculated.

RESULTS

Our primary research question explores how different inputs affect the quality of LLM-generated test cases across accuracy, bug detection, and code coverage. We use a paired two-sided Mann-Whitney U test

to compare different input sets on each dimension. (where each input is an array representing one of R, CC, IC, R-CC, R-IC and contains the corresponding metric values computed for each problem). We report the pairs with noticeable absolute effect size (≥ 0.2) that are also statistically significantly different (p-value < 0.05).

Models

We experimented with five state-of-the-art LLMs with varying sizes and open-source status:

- **Closed-sourced:** GPT-4o (2024-08-06) and GPT-4o-mini (2024-07-18);
- **Open-sourced:** Llama-3.3-70B, Codestral-22B, and Llama-3.1-8B (all instruction-tuned).

Test Case Quality

Table 1 presents the main results of our study, showing the quality of the test cases generated by five state-of-the-art LLMs across RCs.

Closed-source Models When using closed-source LLMs (GPT-4o and GPT-4o-mini), including both requirements and correct code (R-CC) in the prompt results in the highest accuracy (47.24% and 41.94%, respectively). Providing only correct code (CC) or only requirements (R) produces the next best results. Notably, including only incorrect code in the prompt (IC) results in the lowest accuracy (35.34% and 31.64%, respectively). However, augmenting incorrect code with requirements (R-IC) significantly increases the accuracy by approximately 19.38%. This trend extends to bug detection capability and coverage, where, on average, CC yields the highest scores (e.g., 74.39% bug detection capability and 63.97% code coverage for GPT-4o), while including only incorrect code (IC) yields the lowest scores (e.g., 61.94% bug detection capability and 51.49% code coverage for GPT-4o).

We observe that generating tests using CC is statistically significantly better than using IC alone (with noticeable effect size) indicating that when given only code, the models assume that it is correct and do not focus on the actual natural language text in the code—also seen from IC having the lowest bug detection capability (e.g., the variable name `add_a_b` in the expression `add_a_b = a*b` likely makes no difference to the model even though we notice a probable logical error at a glance).

Model	R	CC	IC	R-CC	R-IC
Accuracy					
GPT-4o	43.54	α 47.07	$\alpha\beta$ 35.34	β 47.24	42.25
GPT-4o-mini	38.23	γ 41.87	γ 31.64	41.94	37.70
Llama-3.3-70B	16.07	22.93	14.93	21.19	16.36
Codestral-22B	11.88	18.01	14.92	15.18	13.09
Llama-3.1-8B	δ 2.86	11.94	$\delta\epsilon\zeta$ 10.56	ϵ 3.37	ζ 2.77
Bug Detection Capability					
GPT-4o	69.39	74.39	61.94	72.09	68.14
GPT-4o-mini	67.36	72.76	60.29	70.55	65.61
Llama-3.3-70B	23.48	34.97	22.26	32.61	26.52
Codestral-22B	19.27	26.67	20.95	24.42	20.97
Llama-3.1-8B	2.45	6.85	3.57	3.03	1.97
Coverage					
GPT-4o	60.4	η 63.97	η 51.49	62.82	59.23
GPT-4o-mini	56.79	59.18	48.39	58.88	54.58
Llama-3.3-70B	23.05	32.10	21.69	31.35	26.11
Codestral-22B	18.60	25.09	19.84	23.90	20.89
Llama-3.1-8B	θ 3.14	11.86	$\theta\iota\kappa$ 8.95	ι 3.96	κ 3.28

TABLE 1: Accuracy, bug detection capability, and code coverage using two closed-source and three open-source models on different research cases including requirements (R), correct code (CC), incorrect code (IC), requirement with correct code (R-CC), and requirement with incorrect code (R-IC). Note that the results of the research cases containing the three incorrect codes are combined into one by averaging their results for each problem. The best results are marked in bold. For each model and metric, we perform paired double-sided Mann Whitney U Test with a significance value (p-value) of 0.05 between each pair of research cases and find the effect size for the same. The pairs having absolute effect size ≥ 0.20 are marked with the same Greek letters. We noted that the pairs having noticeable effect size are subsets of the pairs that have a statistically significant difference. Do note that there are other pairs which have statistically significant differences but effect size < 0.20 .

When given only incorrect code, models assume it is correct and generate tests to validate it. In closed-source models, combining requirements with incorrect code slightly lowers performance compared to using requirements alone, as the incorrect code introduces contradictions and misleads the model.

We observe that the range of accuracy for GPT-4o when providing only code (correct or incorrect) is [35.34%, 47.07%], but augmenting code with requirements improves the range to [42.25%, 47.24%]. The same enhancement pattern appears for GPT-4o-mini and repeats for bug detection capability and code coverage metrics as well, indicating that adding requirements to code helps improve the overall quality of the test suite. Furthermore, the results for generating tests using only requirements (R) fall within the ranges of R-IC and R-CC.

Without knowing whether the given code is correct, generating test cases using only requirements or augmenting code with requirements are the most effective approaches.

Test cases generated using only IC are statistically significantly different from those using R-IC. Since the accuracy of R-IC surpasses that of IC, this further confirms that requirements help generate higher quality test cases. A similar trend is observed for bug detection capability and code coverage.

Augmenting incorrect code with requirements helps detect more bugs and increases code coverage.

Open-source Models Test generation using only correct code yields the best accuracy (22.93%, 18.01%, and 11.94% for Llama-3.3-70B, Codestral-22B, and Llama-3.1-8B, respectively). For Llama-3.3-

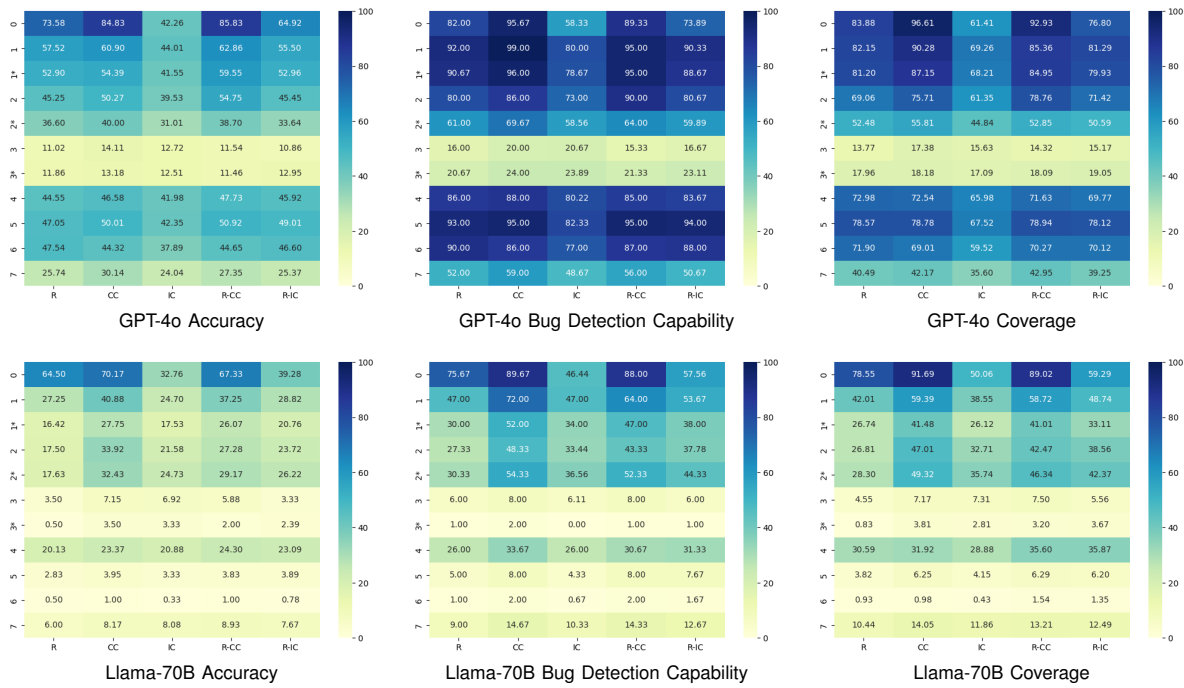


FIGURE 2: Heatmaps showing accuracy, bug detection capability, and code coverage across different difficulty levels and Research Cases (RCs) for the best closed-source model, GPT-4o, and the best open-source model, Llama-3.3-70B. The RCs under consideration are requirement (R), correct code (CC), incorrect code (IC), requirement and correct code (R-CC), and requirement and incorrect code (R-IC). Note that 0, 1, 1*, 2, 2*, 3, 3*, 4, 5, 6, 7 are progressively increasing difficulty levels (0 representing the lowest difficulty level) as determined by CodeChef.

70B, adding requirements to both correct and incorrect code results in the next two best accuracies (21.19% and 16.36%). Similar to closed-source models, generating test cases using only incorrect code yields the lowest accuracy. We observe that passing requirements to Codestral-22B and Llama-3.1-8B negatively impacts accuracy.

For bug detection capability and coverage, Llama-3.3-70B and Codestral-22B do the best using CC and R-CC, and the worst when using IC as seen in Table 1. Llama-3.1-8B obtains the highest bug detection and coverage using CC, lowest bug detection using R-IC and the lowest coverage using R.

Therefore, for Llama-3.3-70B, incorporating requirements into IC improves test quality, though not significantly. Adding requirements to CC does not enhance the test quality and may even degrade it.

For Codestral-22B and Llama-3.1-8B, adding requirements to code perplexes the model and results in similar or lower accuracy. Using only requirements performs worse than using IC.

Smaller models ($\leq 22B$ parameters) struggle to infer specifications and input/output constraints from natural language, compared to using only code.

Difficulty Level

Figure 2 visualizes the metrics across various problem difficulty levels in our dataset. A general decrease in value across all the metrics is observed. This indicates that as the complexity of the problems grows, the quality of the generated test cases decreases. However, we also observe some irregularities in this overall trend, likely due to differences in pretraining data and optimization strategies for each LLM.

Using Mann-Whitney test to analyze differences between the easiest problems (difficulty levels 0-1) and the most difficult problems (levels 6-7) reveals notable patterns. For the easiest problems, adding requirements to code (both, correct and incorrect) or just using requirements, compared to incorrect code resulted in statistically significant differences with substantial effect size (≥ 0.30) between them. This indicates

that incorporating requirements into code or simply using requirements substantially improves test quality. In contrast, for the most difficult problems, although the same comparisons were statistically significant, the effect sizes were small. This suggests that the practical impact of prompt information diminishes as problem difficulty increases. Similar trends were observed across all evaluation metrics for both GPT-4o and Llama 70B.

When the given task is difficult, using more inputs does not help much likely due to model's limitation.

Impact of Model Size

Larger models benefit from both code and requirements, improving test case quality, while smaller models struggle with natural language and perform better using only code.

As model size decreases, the quality of the generated test cases also decreases.

IMPLICATIONS

- 1) Each LLM exhibits distinct strengths and weaknesses influenced by its size, architecture, and pretraining dataset. Our results show that test quality generally decreases with model size. Developers must make informed decisions when selecting an LLM for test generation, considering task complexity and input context. They may need to adjust prompting strategies and provide complete context to optimize performance.
- 2) Requirements play a critical role in generating high-quality test cases, especially since developers cannot always be certain of the correctness of the code under test. Larger models effectively synthesize information from both code and requirements to produce higher-quality test cases. In contrast, smaller models may struggle, with their performance heavily dependent on the complexity of the provided requirements.
- 3) Depending on the selected model, optimizing for both test coverage and bug detection may not be feasible. Developers often need to prioritize one and subsequently address the other based on the codebase and development stage, interpreting both metrics cautiously as indicators of test quality. If both metrics are desired simultaneously, top-performing models like GPT-4o are recommended.

- 4) In real-world software development, the effectiveness of LLM-based test generation depends on the availability of high-quality requirements and correct code. While correctness of added code can't be guaranteed regardless of codebase maturity, having clear requirements helps. Mature codebases offer the added benefit of using existing, likely correct code as context, and may enable requirement generation via LLMs—an interesting direction for future research.

CONCLUSION

We present an empirical study that investigates how different inputs affect the quality of generated test cases across five state-of-the-art LLMs. Our observations reveal that well-defined requirements significantly enhance test quality. However, overly rigid input/output formatting requirements can degrade test quality for smaller models rather than improving it. Overall, requirements—essentially natural language descriptions of desired outcomes—are a powerful and widely applicable way to unlock greater value from LLMs.

Developers can boost their productivity with LLMs for test generation by choosing the right model for their specific needs. However, instead of blindly trusting LLMs' responses, developers should critically analyze outputs for potential errors, i.e., positioning LLMs as valuable assistants while maintaining essential human oversight.

We acknowledge that competitive programming, as represented by the CodeChef dataset, may differ from real-world software development. The problems in such datasets may lack the complexity, diversity, and context-driven aspects that characterize real-world software development tasks.

Additionally, our work points to several promising research directions: (1) An interesting idea would be the use of a feedback loop, where the model uses some information it generates to iteratively enhance its responses. However, this lies outside the scope of our current work since real-world software lacks the ground truth regarding the correctness of underlying code or the generated test cases (another research problem), making the implementation of an effective feedback mechanism challenging without additional signals. Our framework uses a dataset that has information about correctness of the code, a luxury that is often not available in the real-world. A promising avenue, however, could involve leveraging other models to compare generated outputs against external signals like meeting notes, documentation, or other development artifacts, to help the model refine its responses more effectively.

(2) exploring how the prompt structure (like code/requirements length and complexity) affect test case quality (3) analyzing whether reasoning-enabled LLMs outperform non-reasoning LLMs; (4) model finetuning to generate code-correctness agnostic bug-revealing tests; (5) improving smaller models using model distillation. Future work can reuse our evaluation framework to assess the effectiveness of new LLM-based test generation techniques.

REFERENCES

1. D. Kumar and K. K. Mishra, "The impacts of test automation on software's cost, quality and time to market," *Procedia Computer Science*, vol. 79, pp. 8–15, 2016.
2. M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.
3. C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *International Conference on Software Engineering*, 2023, pp. 919–931.
4. J. A. Pizzorno and E. D. Berger, "CoverUp: Coverage-guided LLM-based test generation," *arXiv preprint arXiv:2403.16218*, 2024.
5. L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in *International Conference on Software Engineering*, 2014, pp. 435–445.
6. "Writing tests with GitHub Copilot," <https://docs-internal.github.com/en/copilot/using-github-copilot/guides-on-using-github-copilot/writing-tests-with-github-copilot>.
7. Q. Team, "Writing Unit Tests Best Practices," <https://www.qodo.ai/blog/best-practices-for-writing-unit-tests/>, Apr. 2023.
8. N. S. Mathews and M. Nagappan, "Test-driven development and LLM-based code generation," in *Automated Software Engineering*, 2024, pp. 1583–1594.
9. D. Huang, J. M. Zhang, M. Harman, M. Du, and H. Cui, "Measuring the influence of incorrect code on test generation," *arXiv preprint arXiv:2409.09464*, 2025.
10. S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: a survey," *Software Testing, Verification and Reliability*, vol. 22, no. 2, pp. 67–120, 2012.

Saarang Agarwal is a Master's student at the University of Waterloo, Waterloo, ON, N2L 3G1, Canada.

His research interests focus on enhancing the productivity of software engineers using machine learning. Agarwal received his Bachelor in computer engineering from the University of Waterloo. Contact him at saarang.agarwal@uwaterloo.ca.

Pengyu Nie is an Assistant Professor at the University of Waterloo, Waterloo, ON, N2L 3G1, Canada. His research interests focus on AI for software development, testing, and maintenance. Nie received his Ph.D. in electrical and computer engineering from the University of Texas at Austin. He is a member at IEEE. Contact him at pynie@uwaterloo.ca.

Meiyappan Nagappan is an Associate Professor at the University of Waterloo, Waterloo, ON, N2L 3G1, Canada. His research interests focus on empirical software engineering to address software development concerns and currently researches the impact of large language models on software development. Nagappan was a postdoctoral fellow at the Software Analysis and Intelligence Lab at Queen's University and received his Ph.D. in computer science from North Carolina State University. He is a member at IEEE. Contact him at mei.nagappan@uwaterloo.ca.